

DOCUMENTACIÓN DE PROYECTO · PDV / POS

Manual técnico — PDV (Punto de Venta)

Descripción funcional, arquitectura, requerimientos de instalación, diagramas UML y de base de datos, y guías de diagnóstico, mantenimiento y evolución.

Actualizado 2026-10-02

Partes 1-33 mergeadas

Partes 34-39 pendientes

Backend 170 tests

Android 437 tests

Alembic 0001-0013

Room v8

Estado al 2026-10-02. Generado desde docs/manual-tecnico.md (sha256 76583a0818b7).

Contenido

1. Descripción funcional

- 1.1 Módulos
- 1.2 Modos de backend (BackendMode)

2. Arquitectura

- 2.1 Vista de contexto / despliegue
- 2.2 App Android — capas
- 2.3 Backend FastAPI — capas
- 2.4 Motor de sincronización

3. Modelo de datos (ER)

- 3.1 Campos comunes
- 3.2 Diagrama entidad-relación
- 3.3 Notas de integridad
- 3.4 Room (dispositivo)

4. Diagramas UML

- 4.1 Componentes (visión conjunta)
- 4.2 Clases — patrón repositorio ModeAware (representativo)
- 4.3 Secuencia — login en modo REMOTO
- 4.4 Secuencia — venta en modo REMOTO (con idempotencia y decremento de stock)
- 4.5 Secuencia — resolución de conflicto de sincronización
- 4.6 Secuencia — acción del asistente de IA
- 4.7 Despliegue (Docker Compose)

5. Requerimientos e instalación

- 5.1 Backend — vía Docker (recomendado)
- 5.2 Backend — manual (iteración rápida en debug)
- 5.3 Android
- 5.4 Conectividad dispositivo físico -> backend
- 5.5 Asistente de IA (opcional)
- 5.6 Secretos y despliegue no-local

6. Configuración

- 6.1 Backend — variables de entorno
- 6.2 Android — DataStore (configuración)
- 6.3 Versionado de la app

7. Seguridad

8. Referencia de API

9. Operación y diagnóstico

- 9.1 Health y estado

- 9.2 Logs del backend
- 9.3 Logs / auditoría del dispositivo
- 9.4 Inspeccionar Room en el dispositivo
- 9.5 Problemas frecuentes y causa raíz

10. Mantenimiento

- 10.1 Estructura del repositorio
- 10.2 CI
- 10.3 Pruebas
- 10.4 Migraciones y schema-parity
- 10.5 Añadir o cambiar un endpoint
- 10.6 Convenciones (resumen)
- 10.7 Comandos slash del proyecto

11. Evolución y deuda técnica conocida

- 11.1 Estado del plan (docs/PLAN.md)
- 11.2 Detalle — Parte 39: primer release productivo (pendiente)
- 11.3 Seguimiento del motor de sync (Parte 32)
- 11.4 Backlog (docs/PLAN.md)
- 11.5 Diferidos explícitos
- 11.6 Fortalezas a preservar (revisión 2026-08-30, ampliada con la Parte 32)

12. Glosario

Documento de referencia para el equipo de *application management*: descripción funcional, arquitectura, requerimientos de instalación, diagramas UML y de base de datos, y guías de diagnóstico, mantenimiento y evolución.

- **Código de proyecto:** `POS` (Jira) / `PDV` (repo).
 - **Estado a la fecha de este manual (2026-10-02):** Partes 1-33 implementadas y mergeadas (archivadas en `docs/PLAN-historico.md`), incluyendo el motor de sincronización diferida (Parte 32) y este propio manual (Parte 33). Partes 34-39 (convergencia del inventario, sucursal estable, seguridad pre-release, integridad del backend, calidad menor y primer release productivo) pendientes, sin claves de Jira todavía; salen de la revisión pre-release de `docs/review_code.md`. App corriendo en dispositivo físico contra backend dockerizado en LAN.
 - **Fuentes de verdad vivas** (este manual las resume, no las reemplaza):
 - `CLAUDE.md` — arquitectura estable, convenciones, comandos.
 - `docs/api-contract.md` — contrato de endpoints REST.
 - `docs/schema-pos.json` — esquema de datos compartido (Room + Alembic).
 - `docs/PLAN.md` — trabajo pendiente (Partes 34-39 + Backlog).
 - `docs/PLAN-historico.md` — bitácora cerrada de las Partes 1-33.
 - `docs/review_code.md` — revisión pre-release del 2026-09-28/29 (origen de las Partes 34-39). La revisión anterior (2026-08-30, origen de las Partes 21-30) se archivó como `docs/review_code_2026-09-28_1816.md`.
 - `docs/ALCANCE-DOCS.md` — manifiesto de alcance de este manual y de sus derivados HTML/PDF, mantenidos con `/docs-sync` (§10.7).
-

1. Descripción funcional

PDV es una aplicación de punto de venta para comercios con varias sucursales. Funciona **completamente offline** (Kotlin + Room) y puede **sincronizarse** de forma diferida con un backend FastAPI compartido entre sucursales. Incluye un **asistente de IA opcional** (DeepSeek / OpenAI / OpenRouter) para consultas y actualizaciones asistidas.

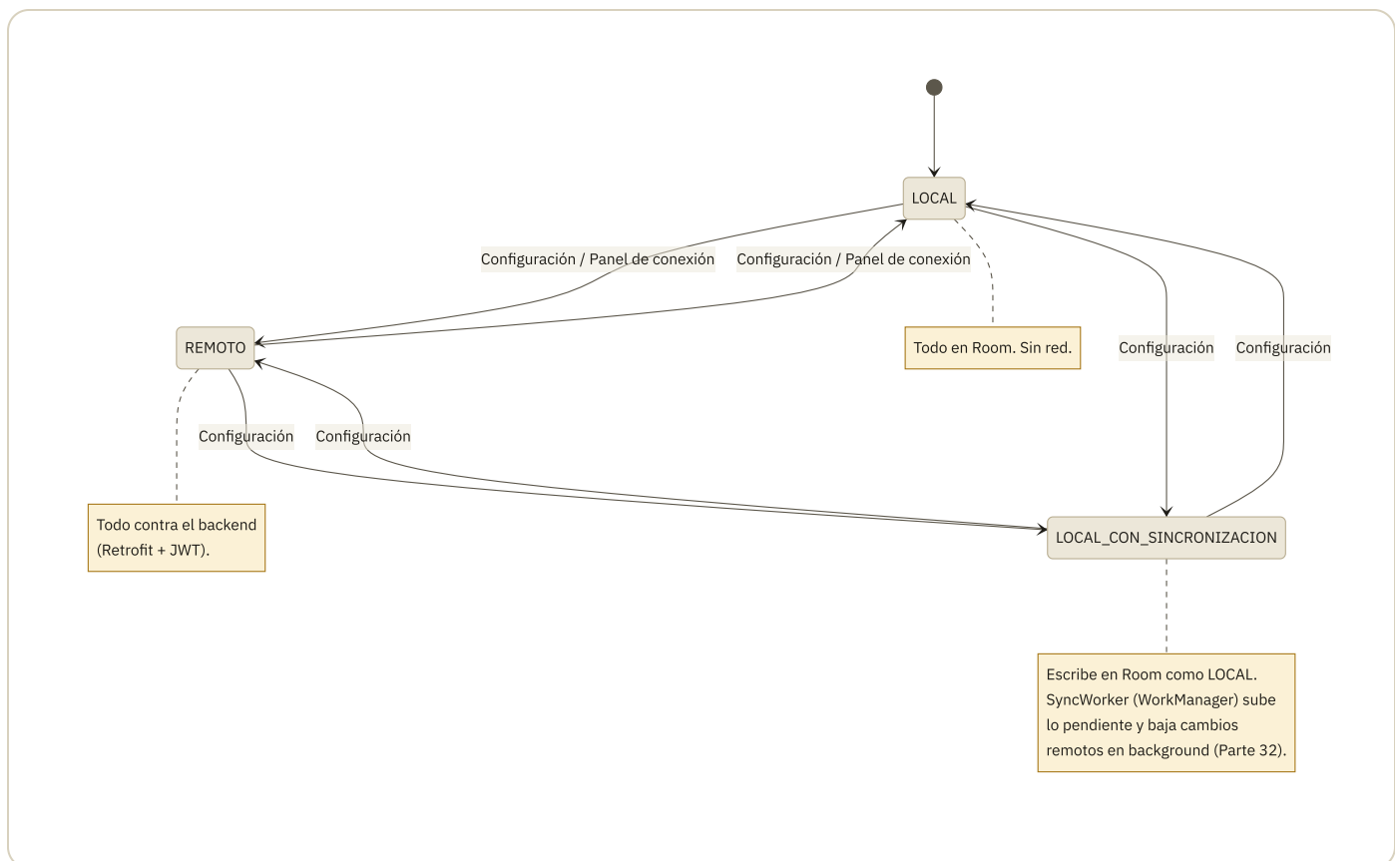
1.1 Módulos

Módulo	Pantalla(s)	Qué hace	Persistencia
Login / Sesión	LoginScreen , PanelConexionBottomSheet	Autenticación por usuario/contraseña; panel de conexión accesible sin sesión (cambiar modo LOCAL/REMOTO, esquema/host/puerto, "Probar conexión").	Sesión en memoria (SessionManager); conexión en DataStore.
Venta de mostrador	VentaScreen	Alta de venta con líneas, escaneo de código de barras (CameraX + ML Kit), método de pago efectivo/tarjeta, cálculo de cambio, ticket PDF e impresión. Decrementa inventario y genera movimiento de salida.	ventas , venta_detalle , inventario , movimientos .
Entrada de mercancía	EntradaScreen	Recepción de mercancía de artículo existente o alta de artículo nuevo. Incrementa inventario (delta aditivo) y genera movimiento de entrada.	movimientos (tipo entrada) , articulos , inventario .
Inventario	InventarioScreen , ImportarCatalogoScreen	Consulta paginada con búsqueda (nombre/SKU/código de barras), ajuste de atributos de catálogo + existencia (genera movimiento de ajuste con delta), exportación CSV/Excel, importación de catálogo desde CSV.	articulos , inventario , movimientos .
Caja	CajaScreen	Cortes de caja parciales y finales, retiros de efectivo, cálculo de totales del periodo (ventas, efectivo, tarjeta, retiros, monto esperado), exportación por periodo.	cortes_caja , retiros_efectivo .

Módulo	Pantalla(s)	Qué hace	Persistencia
Devoluciones	DevolucionScreen	Registro de devolución de cliente con líneas (motivo, condición). No toca inventario (gestión con proveedor).	devoluciones , devolucion_detalle .
Usuarios	UsuarioScreen	CRUD de usuarios, asignación de rol, alta/cambio de contraseña (hash bcrypt calculado en el dispositivo).	usuarios .
Roles	RolScreen	CRUD de roles personalizados; modulos_permitidos activa/desactiva pantallas completas. Dos roles de sistema no editables (administrador , encargado_turno).	roles .
Configuración	ConfiguracionScreen	BackendMode , parámetros de conexión remota, sucursal seleccionada, activación y token del asistente de IA, prompt de sistema editable. En LOCAL_CON_SINCRONIZACION : sección "Sincronización" (última sync, pendientes, último error, botón "Sincronizar ahora") y confirmación de pendientes al cambiar de modo (Parte 32).	DataStore (preferencia de dispositivo); token cifrado con Android Keystore.
Revisión de conflictos	RevisionConflictosScreen	Panel solo lectura de sync_conflicts : lista los conflictos de sincronización detectados y su resolución automática.	sync_conflicts .
Asistente de IA + FAQ	AsistenteIaWidget (overlay flotante)	Chat con LLM; responde consultas, ejecuta acciones (alta de artículo, corte parcial, retiro, devolución, exportar/consultar inventario) por los mismos repositorios que las pantallas manuales, con confirmación explícita y chequeo de permiso. FAQ curado con retrieval de 3 niveles y atajos sin conexión (qN , match de texto).	Reutiliza los repositorios de dominio; FAQ empaquetado (faq.jsonl).

Módulo	Pantalla(s)	Qué hace	Persistencia
Logs / Auditoría	— (sin pantalla)	<code>AppLogger</code> escribe archivos <code>.txt</code> locales al dispositivo, una línea por evento con <code>[tipo][timestamp][sucursal]</code> <code>[usuario] mensaje</code>. Categorías: <code>ERROR</code>, <code>WARN</code>, <code>INFO</code>, <code>DB_WRITE</code>, <code>SYNC_CONFLICT</code>, <code>AUTH</code>. Rotación a 5 MB, retención 30 días.	Sistema de archivos del dispositivo (<code>filesDir</code>).

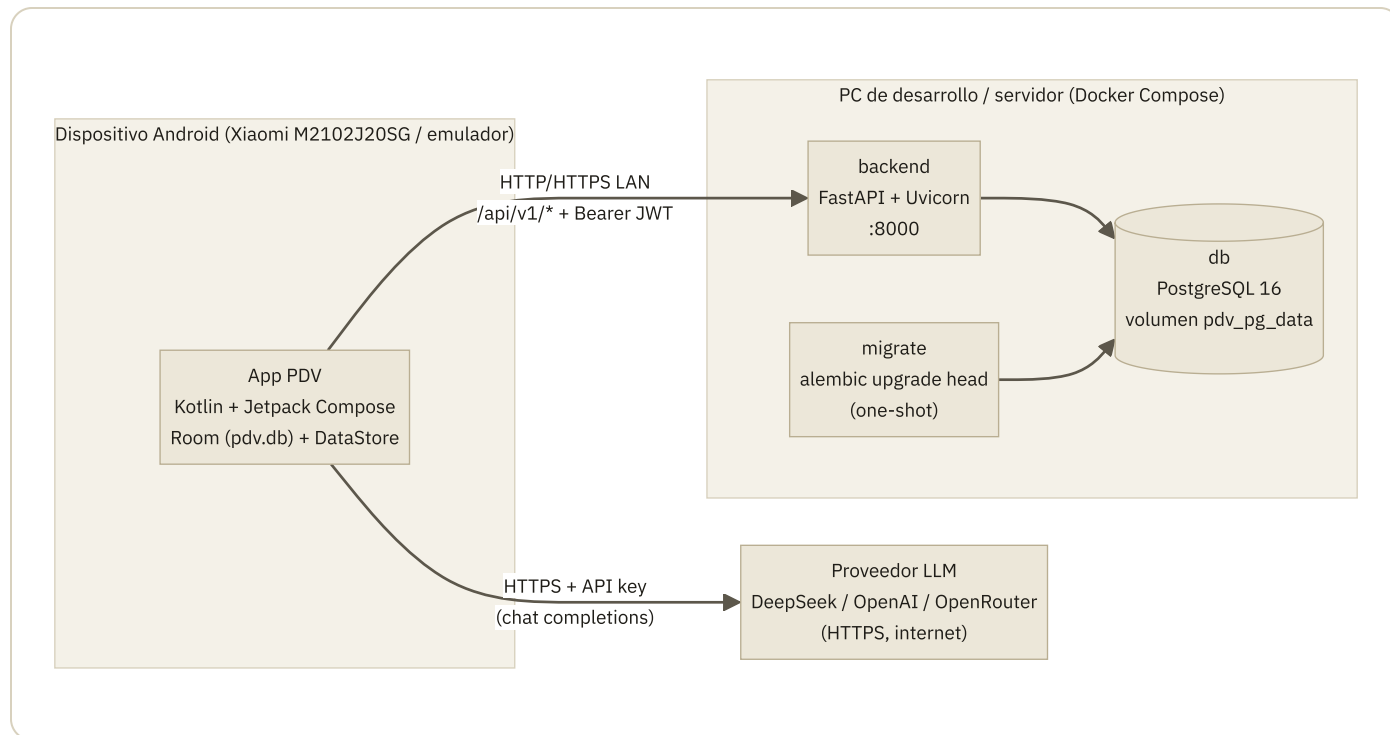
1.2 Modos de backend (`BackendMode`)



- **LOCAL**: la app opera 100% contra Room. No hay tráfico de red de negocio.
- **REMOTO**: cada operación va contra el backend por HTTP con `Authorization: Bearer <JWT>`. La respuesta del servidor es la fuente de verdad.
- **LOCAL_CON_SINCRONIZACION**: escribe en Room igual que LOCAL; el catálogo de sucursales se lee del backend en modo *best-effort* (degrada al catálogo local si falla). Un `SyncWorker` en background (periódico + botón "Sincronizar ahora") sube lo creado offline y baja los cambios remotos de entidades transaccionales, reusando la idempotencia por `local_id` de la Parte 23 para no duplicar (detalle en §2.4).

2. Arquitectura

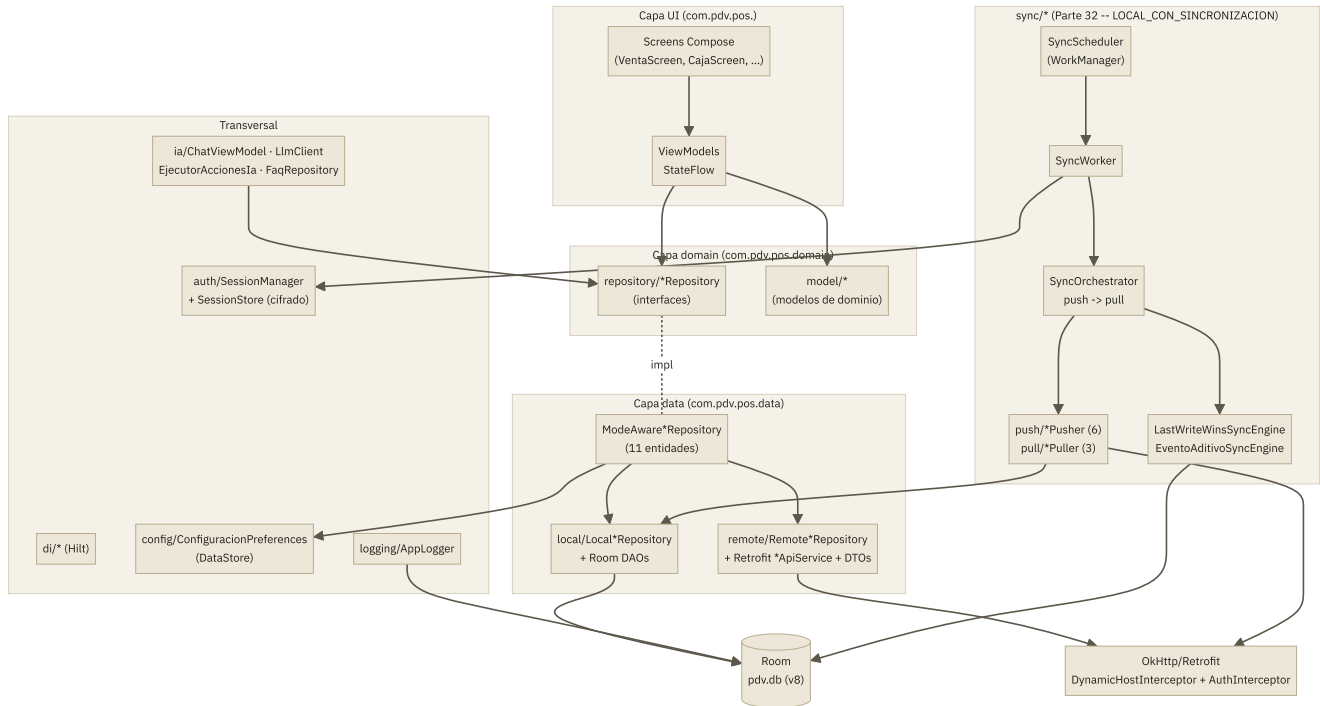
2.1 Vista de contexto / despliegue



- El dispositivo alcanza el backend por **Wi-Fi en la misma subred** (IP LAN de la PC + puerto 8000) o, como alternativa, por **USB con `adb reverse`** (`localhost:8000`).
- El backend se publica en `0.0.0.0:8000` dentro del contenedor; el acceso desde otros equipos de la LAN requiere una regla de firewall inbound (ver §5).
- El asistente de IA llama directo al proveedor LLM desde el dispositivo; el backend no interviene en ese camino.

2.2 App Android — capas

Arquitectura **MVVM + offline-first** con separación estricta `domain / data.local / data.remote`. Paquete raíz `com.pdv.pos`.



Reglas de arquitectura (obligatorias, ver **CLAUDE.md** §3):

1. **domain/** no conoce Room ni Retrofit: los ViewModels dependen solo de las interfaces ***Repository**.
2. **data/local/** **no importa nada de data/remote/**. El modo LOCAL debe compilar y funcionar aislado de la red.
3. La implementación inyectada de cada ***Repository** es siempre el **ModeAwareRepository**, que resuelve local vs. remoto leyendo **BackendMode** de DataStore en cada llamada.
4. Nunca se lanzan excepciones sin capturar desde la capa de red: se usa **ApiResponse.Success/Error** o **Result<T>**.
5. **BackendMode**, parámetros de conexión y sucursal seleccionada son preferencia de dispositivo y viven en DataStore, **sin Room** (excepción documentada). El catálogo de sucursales sí sigue el patrón repositorio/Room.

Patrón **ModeAware*** (ejemplo real, **ModeAwareVentaRepository**):

```

override suspend fun registrarVenta(venta: Venta) {
    when (preferences.deviceConfig.first().backendMode) {
        BackendMode.LOCAL, BackendMode.LOCAL_CON_SINCRONIZACION -> local.registrarVenta(venta)
        BackendMode.REMOTO -> remote.registrarVenta(venta)
    }
}

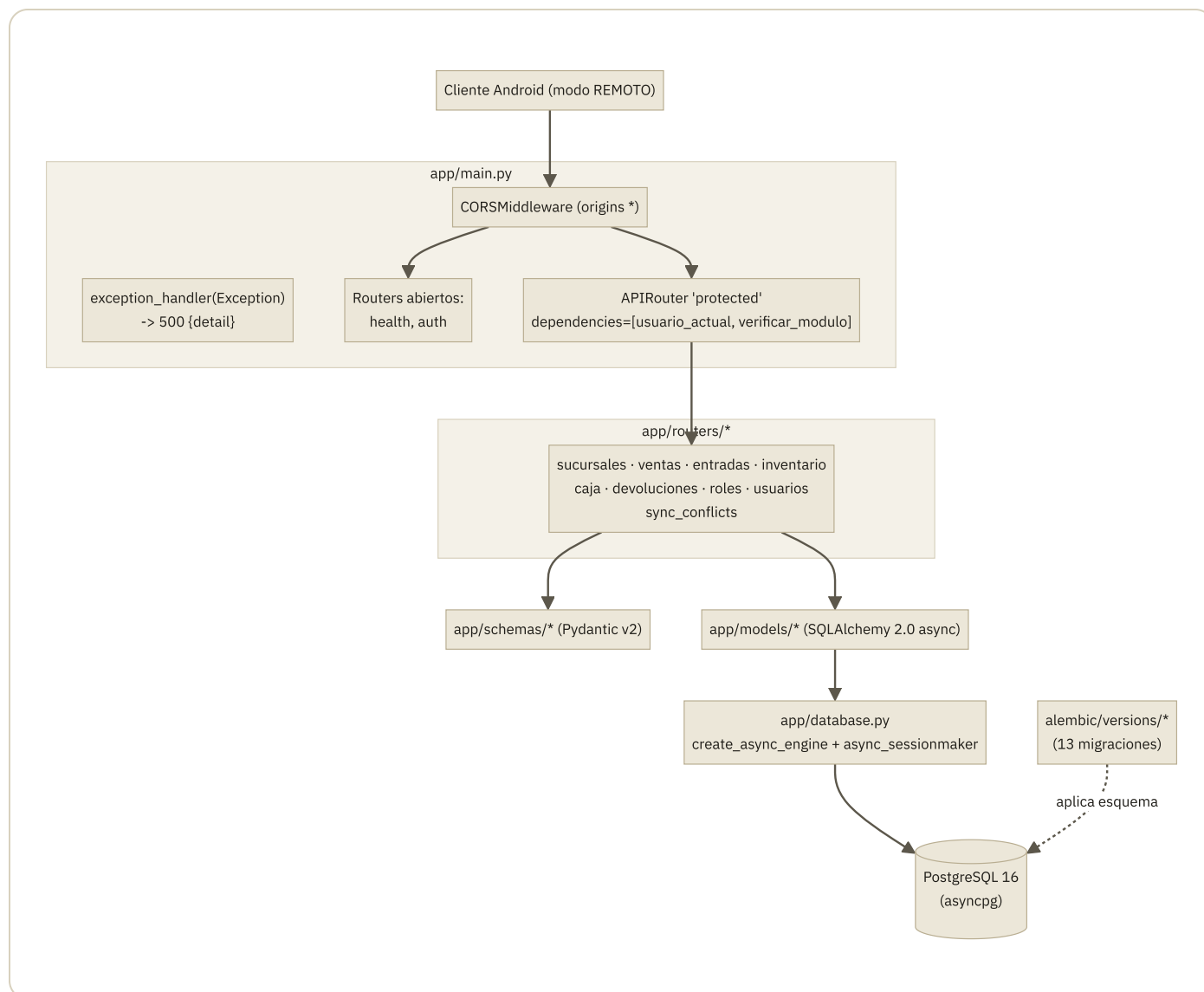
```

```
    inventarioRefreshSignal.emitir()
}
```

Stack Android (`android/gradle/libs.versions.toml`):

Área	Tecnología	Versión
Lenguaje / build	Kotlin / AGP / KSP	2.2.21 / 8.13.2 / 2.2.21-2.0.5
SDK	compileSdk / targetSdk / minSdk	36 / 36 / 26 (Android 8.0)
UI	Jetpack Compose BOM · Material3 · Navigation-Compose	2026.06.01 · — · 2.9.6
DI	Hilt	2.57.2
Sync en background	WorkManager · Hilt-Work (Parte 32)	2.10.1 · 1.2.0
Red	Retrofit · OkHttp · kotlinx.serialization	3.0.0 · 5.4.0 · 1.11.0
Persistencia local	Room · DataStore Preferences	2.8.4 · 1.2.1
Escaneo	CameraX · ML Kit barcode-scanning	1.6.1 · 17.3.0
Cripto	bcrypt (at.favre.lib)	0.10.2
Test	JUnit5 · MockK · kotlinx-coroutines-test · OkHttp MockWebServer	6.1.3 · 1.14.11 · 1.11.0 · 5.4.0
JVM target	17	

2.3 Backend FastAPI — capas



- **Autenticación y autorización** se declaran **una sola vez** a nivel del router `protected` (`app/main.py`):
 `Depends(usuario_actual)` valida el JWT y carga el `Usuario` (con su `rol` vía `selectinload`);
 `Depends(verificar_modulo)` aplica el chequeo de módulo a las escrituras.
- `health` y `auth` quedan fuera del router protegido (rutas abiertas).
- El `exception_handler` global convierte cualquier excepción no prevista en `500 {"detail": "error interno del servidor"}` y la loguea a nivel `ERROR` (sin traceback al cliente). No intercepta `HTTPException` ni `422`.
- **Sin ORM como `response_model`**: los routers devuelven esquemas Pydantic de `app/schemas/`.
- **Montos y cantidades** viajan como *string JSON* en el contrato para no perder precisión decimal; en Postgres son `Numeric(12,2)` / `Numeric(14,3)`.

Stack backend (backend/pyproject.toml , uv.lock):

Área	Tecnología	Versión
Runtime	Python	>=3.11 (3.11.15 gestionado por uv)
Framework	FastAPI · Uvicorn[standard]	0.141.1 · 0.52.4
ORM / migraciones	SQLAlchemy 2.0 (async) · Alembic	2.0.52 · 1.19.1
Driver DB	asyncpg	0.31.0
Cripto / auth	bcrypt · PyJWT	5.0.0 · 2.13.0
Gestor de dependencias	uv	0.10.12
Test	pytest · pytest-asyncio · httpx	9.1.1 · 1.4.0 · 0.28.1
Base de datos	PostgreSQL	16

2.4 Motor de sincronización

Diseño *last-write-wins* + eventos aditivos (android/.../sync/ , docs/PLAN.md Parte 6, docs/api-contract.md §12).

- **Entidades sin ambigüedad de cantidad** (precios, usuarios, roles, sucursales): LastWriteWinsResolver compara updated_at ; gana el más reciente.
- **Entidades de cantidad/movimiento** (inventario, ventas, cortes, devoluciones): EventoAditivoCombiner.combinar(base, deltaLocal, deltaRemoto) — nunca un UPDATE cantidad = X directo, siempre suma de deltas con signo. El backend aplica el mismo principio (inventario.cantidad = cantidad_actual ± delta , con with_for_update() para evitar carreras).
- **Auditoría:** toda divergencia de updated_at se registra en sync_conflicts (aunque se resuelva automáticamente) y se loguea con LogType.SYNC_CONFLICT .
- **Idempotencia de los POST de sync** (Parte 23 + gap 1 de la Parte 32): ventas , movimientos (entradas), cortes_caja , retiros_efectivo y devoluciones tienen UNIQUE(local_id) ; el router hace flush() , captura IntegrityError , hace rollback y devuelve 200 con la fila ya persistida en vez de duplicar. sync_conflicts es idempotente por su id generado en el dispositivo.
- **Orquestación del push/pull en background** (android/.../sync/push/ , sync/pull/ , Parte 32): SyncScheduler encola un SyncWorker (CoroutineWorker + Hilt) periódico (~15 min, con Constraints de conectividad y backoff exponencial) más un OneTimeWorkRequest desde el botón "Sincronizar ahora" de Configuración — ambos no-op salvo BackendMode == LOCAL_CON_SINCRONIZACION .

`DefaultSyncOrchestrator` corre primero el **push**, un `EntityPusher` por entidad, en el orden entradas → ventas → ajustes de inventario → cortes → retiros → devoluciones (para que los artículos/ventas creados offline ya tengan `remote_id` cuando otra fila los referencia), y luego el **pull** (`EntityPuller` de `inventario` / `cortes_caja` / `retiros_efectivo`, acotado a la sucursal seleccionada, con el cursor `? updated_since=` que persiste `SyncStateStore` y avanza al `max(updated_at)` recibido).

- **Política de errores del push:** `IOException` / `5xx` reintentan el ciclo completo; `401` corta el ciclo y marca la sesión expirada; un `4xx` no-401 en una fila puntual la descarta (`isSynced = true` sin `remote_id`, para no reintentarla en cada ciclo) salvo que la causa sea un id local aún sin resolver a remoto (ventas/devoluciones con líneas), en cuyo caso la fila se **aplaza** al próximo ciclo en lugar de descartarse.
 - **Autenticación del worker desatendido:** `SessionStore` persiste la sesión (incluido el `accessToken`) cifrada con `TokenCipher` /Android Keystore (mismo patrón que el token de IA, Parte 14) y siembra `SessionManager` al arrancar el proceso, para que el `SyncWorker` pueda adjuntar `Authorization: Bearer` tras un reinicio sin que haya sesión interactiva.
 - **Estado actual:** motor, orquestación y resolución de conflictos existen, están probados (`jvm-tests` con fakes, sin Robolectric) y **verificados de punta a punta en el Xiaomi** (push, pull cross-terminal y cambio de modo con confirmación de pendientes). La revisión pre-release encontró que el inventario no converge entre terminales de una misma sucursal (Parte 34, pendiente) y otros fallos silenciosos (Parte 35, pendiente); ver §11.3.
-

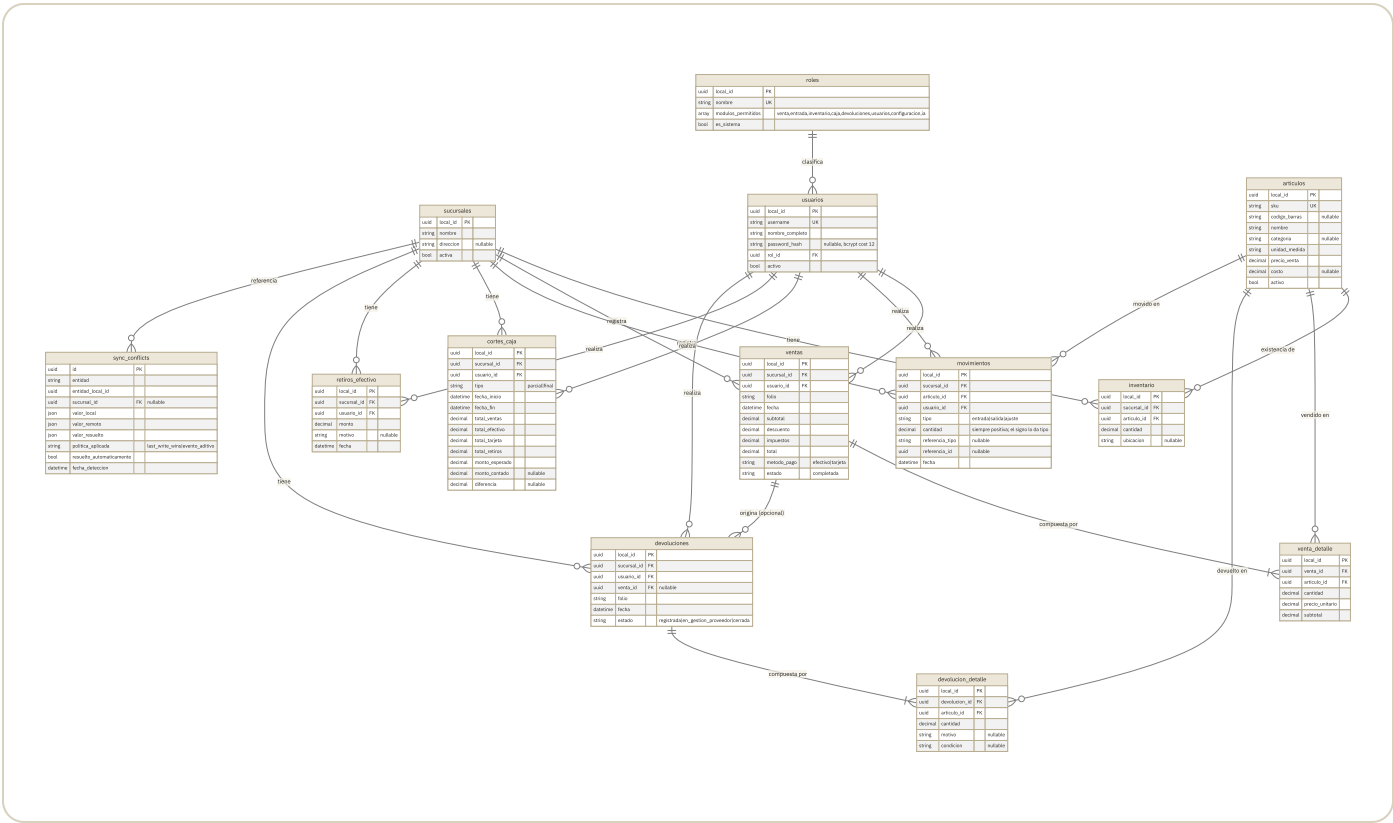
3. Modelo de datos (ER)

Fuente de verdad: docs/schema-pos.json. La compuerta schema-parity exige que el JSON, la entidad Room y la migración Alembic queden alineados entre sí.

3.1 Campos comunes

- **Tracking de sincronización** (toda tabla salvo sync_conflicts): local_id (uuid, PK local en Room), remote_id (uuid, nulo hasta el primer push; en el backend es la PK id), updated_at (ISO-8601 UTC), is_synced (bool), deleted_at (soft-delete; nunca DELETE físico).
- **sucursal_id** (uuid, FK a sucursales): presente en toda entidad transaccional (sucursal_scoped): inventario, ventas, cortes_caja, retiros_efectivo, devoluciones, movimientos, sync_conflicts. No lo llevan articulos, roles, usuarios (globales entre sucursales).
- **Seed**: si no existe ninguna fila en sucursales al arrancar, se crea "Sucursal principal" (backend: migración inicial; Android LOCAL: primer arranque).

3.2 Diagrama entidad-relación



3.3 Notas de integridad

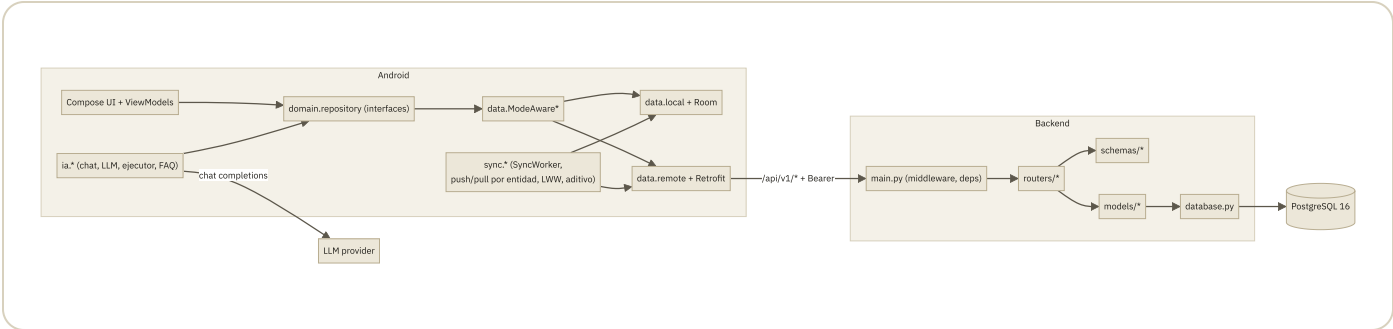
- `venta_detalle` **no tiene FK real** a `articulos` (decisión de la Parte 8); `inventario`, `movimientos`, `devolucion_detalle` **sí** la tienen.
- `retiros_efectivo` **no tiene FK** a `cortes_caja`: se asocian por rango de fecha del lado del dispositivo.
- `sync_conflicts` no participa del ciclo `local_id/remote_id/is_synced/deleted_at`: es el propio registro de auditoría; no se borra (no hay `DELETE`).
- Backend: `ventas`, `movimientos`, `cortes_caja` y `retiros_efectivo` tienen `local_id` `UNIQUE` desde la Parte 23 (migración `0012`); `devoluciones` lo suma en la Parte 32 (migración `0013`, gap 1 de esa Parte) porque su push diferido también lo necesita. En `movimientos` solo las entradas llevan `local_id`; las filas de venta/ajuste lo dejan nulo (`UNIQUE` permite múltiples `NULL`).

3.4 Room (dispositivo)

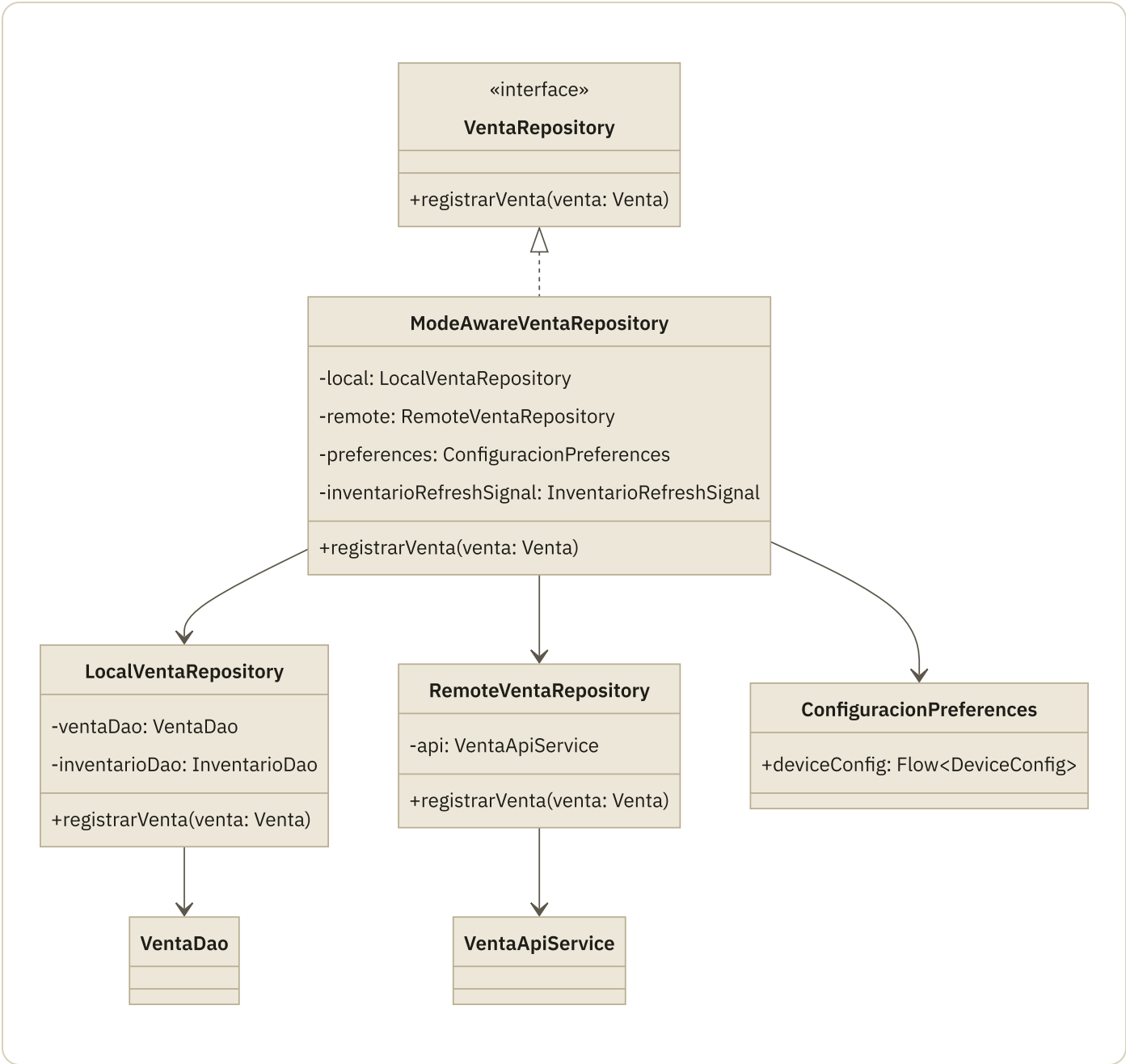
- Base `pdv.db`, `@Database(version = 8, exportSchema = true)`, 13 entidades, 10 DAOs (`PdvDatabase`). Esquemas exportados en `android/app/schemas/`.
 - `Converters` (TypeConverters): `BigDecimal <-> String` (`toPlainString`), `List<modulo> <-> String` con separador U+001F (Unit Separator).
 - `inventario.cantidadNum` (REAL) es columna espejo de `cantidad` (TEXT) para poder `SUM()` / `ORDER BY` numérico en SQL (M-9, migración Room `MIGRATION_7_8`). Toda escritura de `cantidad` debe pasar por `nuevoInventario(...)` / `InventarioEntity.conCantidad(...)` para no desincronizar las dos columnas.
 - Migraciones registradas en `DatabaseModule` (`addMigrations(MIGRATION_7_8)`); la v7 es la línea base con `exportSchema` (Parte 24).
-

4. Diagramas UML

4.1 Componentes (visión conjunta)

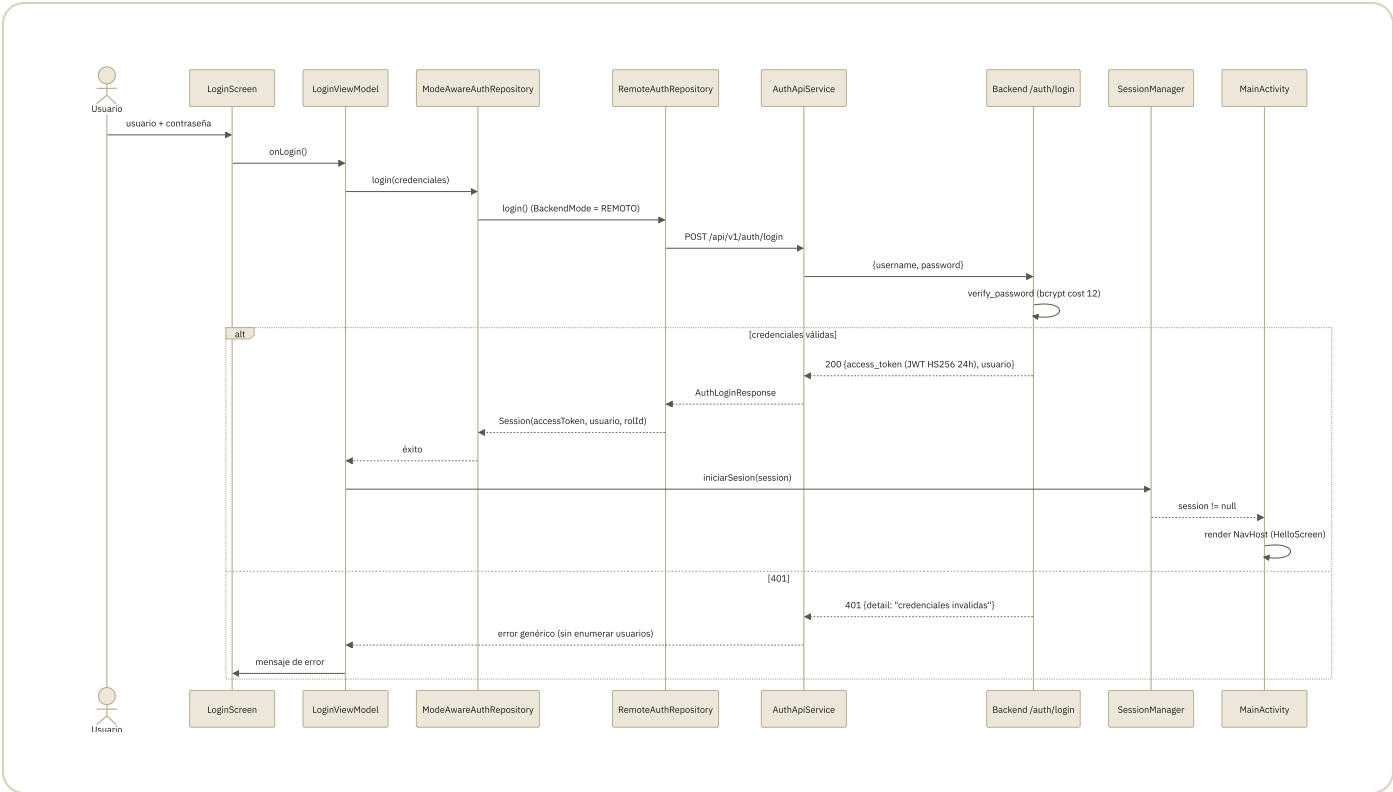


4.2 Clases — patrón repositorio **ModeAware** (representativo)



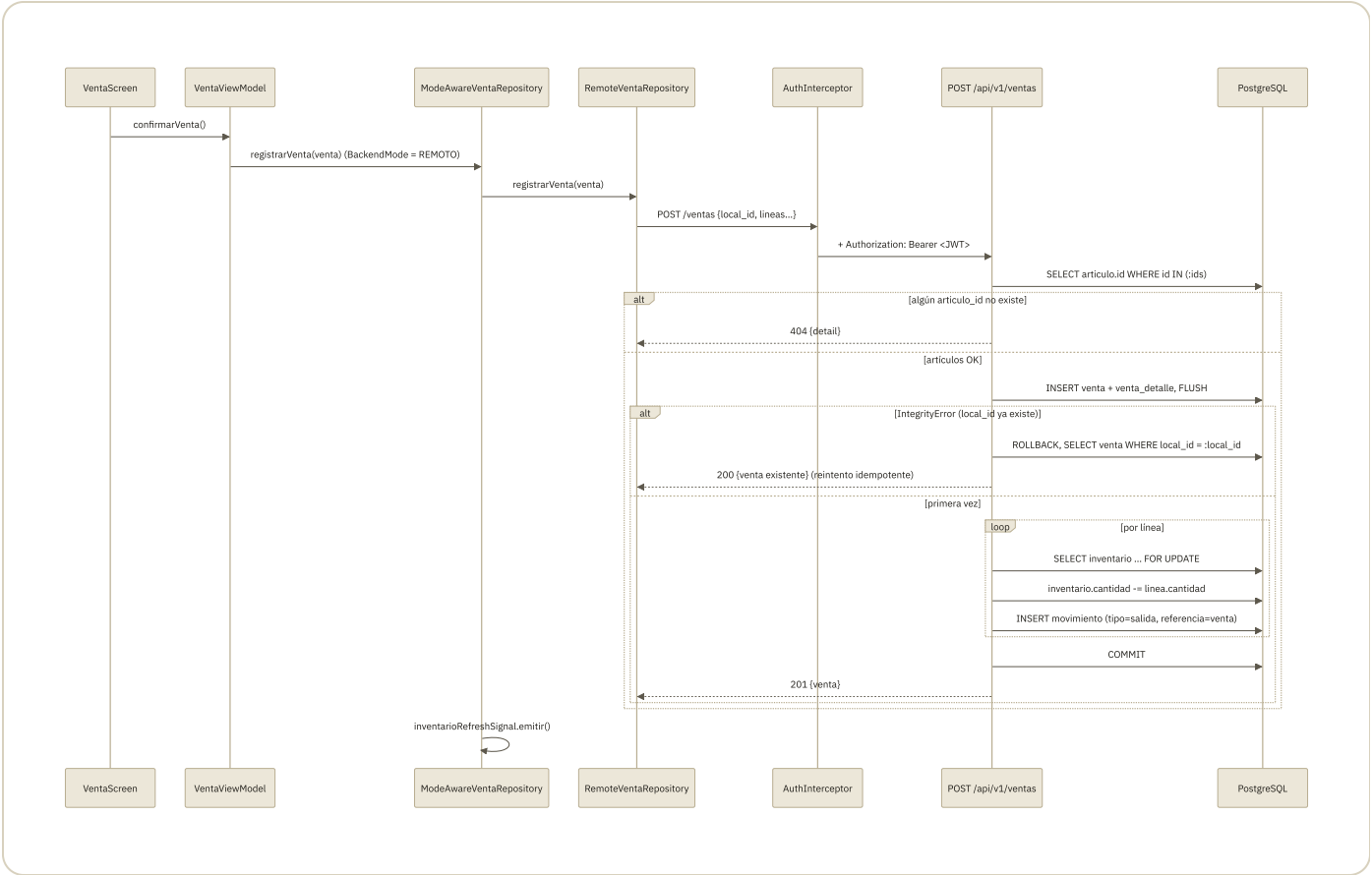
El mismo patrón se repite en las 11 entidades: **Sucursal**, **Venta**, **Entrada**, **Inventario**, **Caja**, **RetiroEfectivo**, **Devolucion**, **Usuario**, **Rol**, **SyncConflict**, **Auth** (todas cableadas en `di/RepositoryModule`).

4.3 Secuencia — login en modo REMOTO



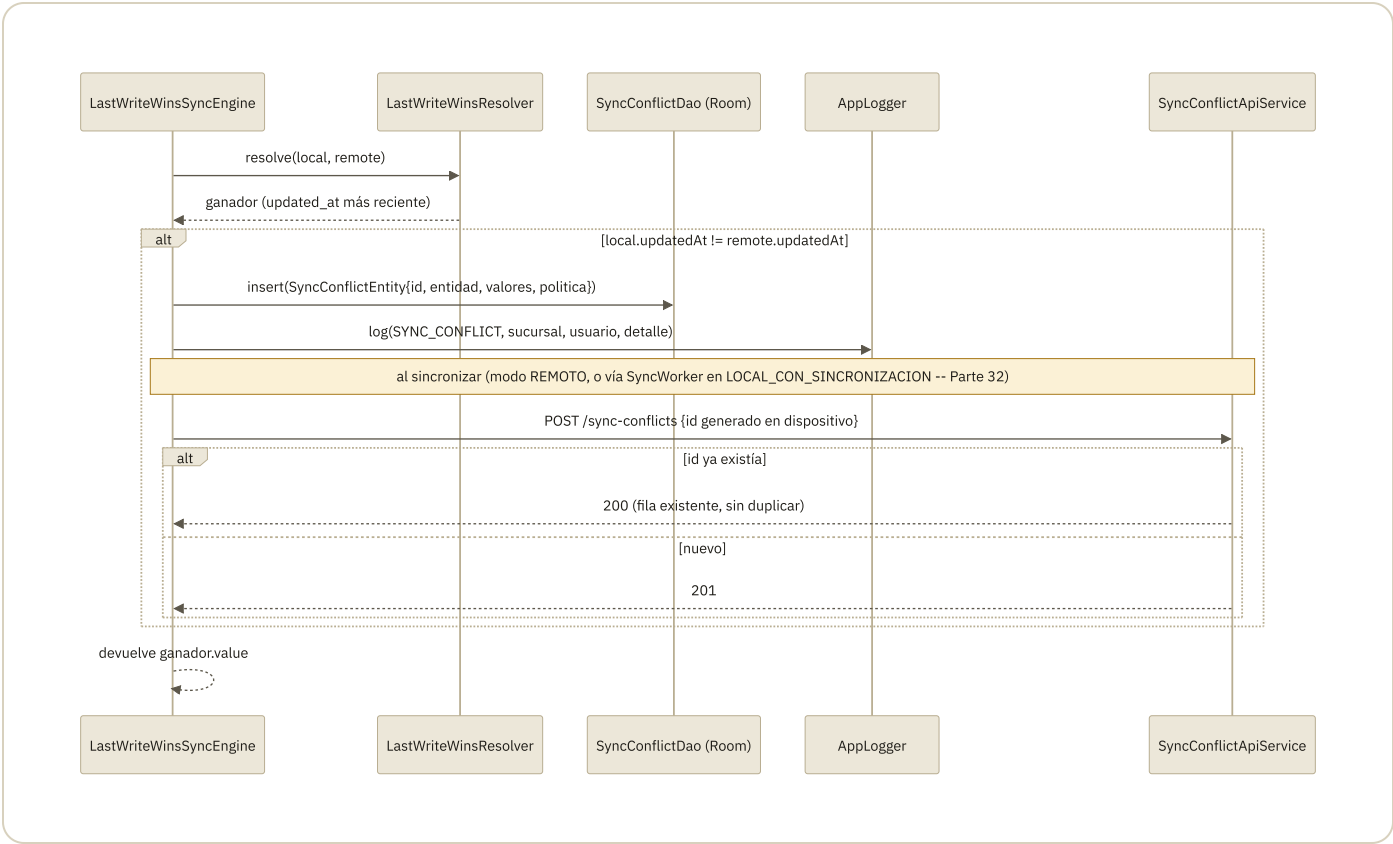
En **modo LOCAL** la rama equivalente va a `LocalAuthRepository`, que valida contra `usuarios / roles` en Room (bcrypt en `Dispatchers.Default`); la sesión resultante no tiene `accessToken`.

4.4 Secuencia — venta en modo REMOTO (con idempotencia y decremento de stock)

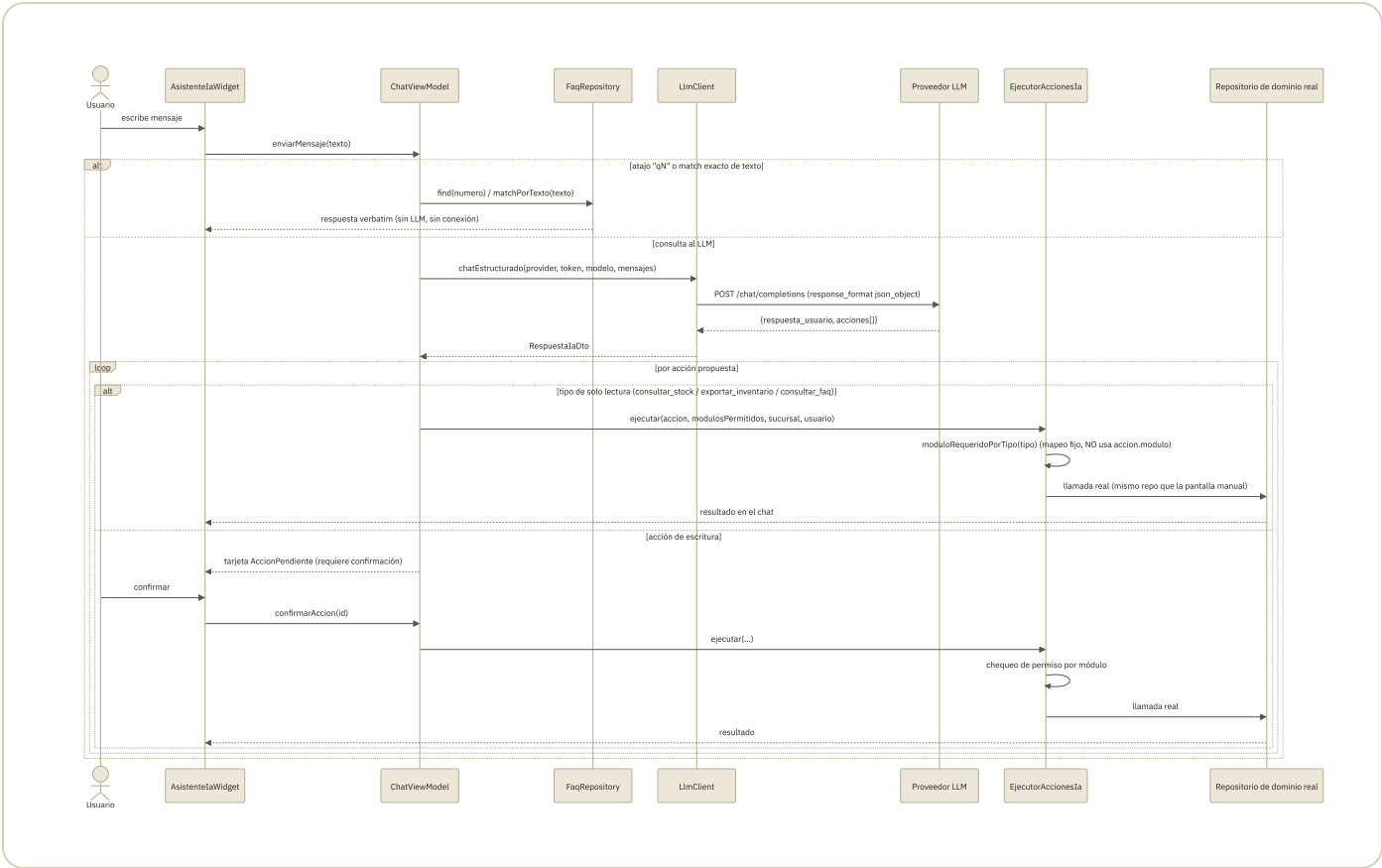


En **modo LOCAL** la operación equivalente ocurre en una transacción Room (`LocalVentaRepository`): inserta `venta` + `venta_detalle` , decrementa `inventario` , inserta `movimiento` , todo con `isSynced = false` .

4.5 Secuencia — resolución de conflicto de sincronización

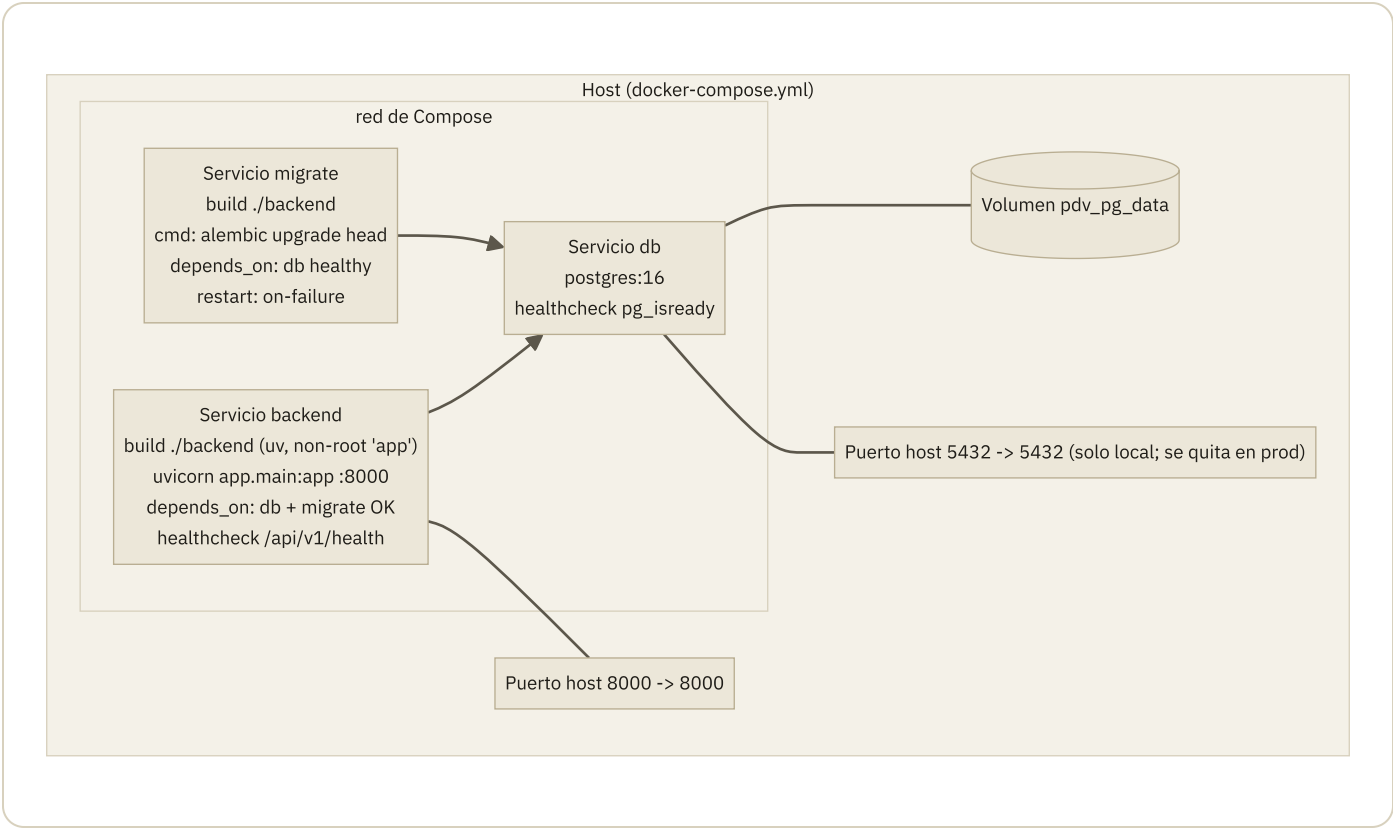


4.6 Secuencia — acción del asistente de IA



Seguridad de la IA: el módulo que autoriza cada acción se deriva **siempre** de `accion.tipo` (mapa fijo en `EjecutorAccionesIa.moduloRequeridoPorTipo()`), nunca del campo `accion.modulo`, porque ambos vienen de la misma fuente no confiable (la salida del LLM). Las 4 acciones de escritura exigen confirmación explícita del usuario en la UI antes de ejecutarse.

4.7 Despliegue (Docker Compose)



5. Requerimientos e instalación

5.1 Backend — vía Docker (recomendado)

Requisitos: Docker + Docker Compose.

```
# Windows
scripts/start-windows.ps1      # docker compose up -d --build
scripts/stop-windows.ps1

# Mac
scripts/start-mac.sh
scripts/stop-mac.sh

# Linux
scripts/start-linux.sh
scripts/stop-linux.sh
```

- Backend disponible en `http://localhost:8000`; health en `GET /api/v1/health -> {"status":"ok","version":"0.1.0"}`.
- El servicio `migrate` aplica todas las migraciones Alembic antes de que arranque `backend`.
- **Rebuild obligatorio tras cambiar código de `backend/app/`:** el `Dockerfile` hornea el fuente (`COPY app ./app`); no hay volumen ni `--reload`. Volver a correr `start-*` (que hace `up -d --build`) es suficiente. `uv run pytest` corre contra el host, **no** contra el contenedor: verde en pytest no implica contenedor actualizado.

5.2 Backend — manual (iteración rápida en debug)

Requisitos: Python 3.11+, `uv` (0.10.12+), un PostgreSQL alcanzable.

```
cd backend
uv sync                                # crea .venv desde uv.lock (incluye grupo dev)
uv run uvicorn app.main:app --reload
uv run alembic upgrade head
uv run pytest
uv run alembic check                  # detecta drift modelo <-> migraciones
```

`backend/requirements.txt` es un **export generado** (`uv export`) que se mantiene solo por compatibilidad externa; no lo consumen ni el `Dockerfile` ni el CI, y no se edita a mano.

5.3 Android

Requisitos:

- Android Studio (o SDK de línea de comandos) con **compileSdk 36, JDK 17**.
- Dispositivo físico con **Android 8.0+ (API 26)** — objetivo de pruebas: **Xiaomi M2102J20SG** — o emulador de fallback.
- Permisos en runtime: `CAMERA` (escaneo de código de barras), `RECORD_AUDIO` (entrada por voz del asistente).

```
cd android
./gradlew assembleDebug          # genera pos-hello-debug-v0.1.apk
./gradlew installDebug           # instala en el dispositivo/emulador conectado
./gradlew testDebugUnitTest      # tests unitarios JVM
./gradlew lint
```

"Definición de hecho" para cambios en `android/`: no basta con compilar o pasar revisión textual; debe quedar instalado y verificado corriendo en el dispositivo físico (o el emulador de fallback).

5.3.1 Hardware y espacio del dispositivo

El proyecto no declara un piso formal de hardware más allá de `minSdk 26`. Los valores de abajo son mínimos prácticos derivados del stack (Jetpack Compose + CameraX + modelo TFLite de ML Kit en el mismo proceso) y de medir los APK ya compilados.

Recurso	Mínimo	Recomendado	Notas
SO	Android 8.0 (API 26)	Android 11+	Validado en Xiaomi M2102J20SG (Redmi Note 10S).
CPU / ABI	<code>armeabi-v7a</code> (ARM 32-bit)	<code>arm64-v8a</code> (ARM 64-bit)	El APK empaqueta las 4 ABIs; en la instalación solo se extrae la nativa de la ABI del dispositivo.
RAM	~2 GB	3-4 GB	Sin floor declarado. Compose, la vista previa de cámara y el modelo de barcode corren en proceso.

Recurso	Mínimo	Recomendado	Notas
Cámara trasera con autofocus	opcional	sí	<code>uses-feature android:required="false"</code> : la app instala sin cámara, pero el escaneo de código de barras queda inhabilitado.
Micrófono	opcional	—	Solo para la entrada por voz del asistente de IA.
Pantalla	teléfono táctil, vertical	—	Diseño Compose pensado para portrait.
Red	ninguna en modo LOCAL	Wi-Fi	Wi-Fi en la misma subred para modo REMOTO / sync. Internet solo para el asistente de IA.

Tamaño de los artefactos actuales (build sin R8, sin ofuscación, sin App Bundle ni ABI splits):

- APK debug: **41,6 MB**. APK release sin firmar: **36,2 MB** (`docs/PLAN.md` Parte 39 registra 36,5 MB en la medición del 2026-09-29).
- DEX (multidex, 17 archivos): ~45 MB sin comprimir en debug — grande porque `isMinifyEnabled = false` (se reduce en la Parte 39).
- Nativas de ML Kit barcode (`libbarhopper_v3.so`): ~4,8 MB para la ABI `arm64-v8a` + ~0,9 MB de modelos `.tflite` empaquetados (sin descarga en runtime).

Espacio en el dispositivo:

Concepto	Estimación	Comentario
Instalación de la app	~90-160 MB	APK almacenado + librería nativa de la ABI extraída + artefactos AOT de ART (oat/vdex). Reservar >= 250 MB libres.
<code>pdv.db</code> (Room)	Catálogo de miles de artículos ~ pocos MB; ~1-2 KB por venta (venta + líneas + movimientos + ajuste de inventario) -> 10 000 ventas ~ 15-25 MB	Sin purga automática (todo es soft-delete). Un dispositivo con años de operación offline necesita una estrategia de archivado/depuración.
Logs (<code>AppLogger</code>)	Pocos MB típico; decenas de MB con logging intenso	Archivos <code>app-log-*.txt</code> de 5 MB máx.; se purgan los de más de 30 días. No hay tope de tamaño total, solo retención temporal.
Tickets PDF y exportaciones CSV/Excel	Pequeño (ticket pocos KB; export escala con el catálogo)	Van a <code>cacheDir/exports/</code> . Son cache: el SO puede evictarlos bajo presión de almacenamiento; la app no los borra por su cuenta.

La Parte 39 (`isMinifyEnabled` + `isShrinkResources` + AAB / ABI splits) reduciría de forma notable el tamaño de descarga e instalación.

5.4 Conectividad dispositivo físico -> backend

Camino principal — Wi-Fi en la misma subred:

1. Teléfono y PC en la misma red Wi-Fi.
2. En la app: panel de conexión (login o Configuración) -> modo **REMOTO**, esquema `http`, IP = IP LAN de la PC (`ipconfig` -> "Dirección IPv4"), puerto `8000`.
3. Regla de firewall de Windows (una sola vez, PowerShell admin):

```
New-NetFirewallRule -DisplayName "PDV backend 8000" -Direction Inbound -Protocol TCP -  
LocalPort 8000 -Action Allow
```

4. Si el router aísla clientes Wi-Fi ("AP/client isolation"), desactivarlo.
5. La IP LAN es DHCP y puede cambiar entre reinicios: reservarla (DHCP estático) o fijar IP estática en la PC.

Alternativa — túnel USB (`adb reverse`): IP `localhost:8000` en la app y en la PC:

```
scripts\install-apk-tunneling.ps1 # installDebug + adb reverse tcp:8000 tcp:8000  
scripts\stop-apk-services.ps1
```

El túnel **no persiste** entre reconexiones de cable, reinicios ni `adb kill-server`; hay que rehacerlo cada sesión.

Cleartext HTTP: habilitado solo en el build de **debug**

(`app/src/debug/res/xml/network_security_config.xml`, `base-config`). El build de release bloquea cleartext por defecto (endurecimiento formal: Parte 39).

5.5 Asistente de IA (opcional)

- En Configuración: activar IA, elegir proveedor (`DEEP_SEEK` / `OPEN_AI` / `OPEN_ROUTER`), pegar el **API key del proveedor** y opcionalmente el modelo.
- El token se cifra con AES-256 respaldado por Android Keystore; en DataStore solo se guarda ciphertext + IV (`AndroidKeystoreTokenCipher`).
- El asistente llama directo al proveedor por HTTPS; no pasa por el backend.

5.6 Secretos y despliegue no-local

- `docker-compose.yml` usa defaults solo-local (`pdv / pdv`, `JWT_SECRET_KEY` de desarrollo) vía `${VAR:-default}`.
 - Para cualquier entorno no-local: copiar `docker-compose.prod.yml.example` a `docker-compose.prod.yml` (gitignoreado) con `JWT_SECRET_KEY` fuerte y credenciales reales de Postgres, y levantar con `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build`.
 - El destino de despliegue productivo (Render / Fly.io / VPS / AWS) está **sin decidir** (`CLAUDE.md` §7 `[TODO]`); se resuelve en la Parte 39).
-

6. Configuración

6.1 Backend — variables de entorno

Variable	Default	Uso
DATABASE_URL	postgresql+asyncpg://pdv:pdv@db:5432/pdv	Conexión async a Postgres (app/database.py , alembic/env.py).
JWT_SECRET_KEY	dev-secret-key-cambiar-en-produccion	Firma HS256 de los JWT (app/security.py). Cambiar antes de exponer fuera de la LAN.

Parámetros fijos en código (app/security.py): JWT_ALGORITHM = "HS256" , JWT_EXPIRES_MINUTES = 60*24 (24 h), bcrypt rounds=12 .

6.2 Android — DataStore (configuracion)

ConfiguracionPreferences sobre DataStore Preferences (sin Room). Claves:

Clave	Tipo	Descripción
backend_mode	LOCAL / REMOTO / LOCAL_CON_SINCRONIZACION	Modo activo. Default LOCAL .
esquema	HTTP / HTTPS	Esquema de la conexión remota (EsquemaConexion). Default HTTP .
ip	String	Host del backend remoto.
puerto	String	Puerto del backend remoto.
sucursal_id_seleccionada	String?	Sucursal activa del dispositivo.

Otras preferencias: IaPreferences (activo, proveedor, modelo, token cifrado), PromptIaPreferences (prompt de sistema editable), SyncStateStore (Parte 32: lastSuccessAtMillis , lastError , cursor updated_since por entidad del pull), SessionStore (Parte 32: sesión del usuario para el SyncWorker desatendido, accessToken cifrado igual que el token de IA).

DynamicHostInterceptor cachea DeviceConfig en un campo @Volatile (sembrado una vez de forma bloqueante, mantenido por un colector) y reescribe esquema/host/puerto de cada request; si no hay IP o el

puerto no es numérico, deja pasar la request con el `BASE_URL` por defecto (`http://localhost:8000/api/v1/`).

6.3 Versionado de la app

- `versionName` SemVer `MAJOR.MINOR.PATCH` (cadena legible del alcance publicado; se sube a mano en el commit de release).
 - `versionCode = MAJOR*10000 + MINOR*100 + PATCH` (entero monotónico, sin colisiones entre APKs distribuidos).
 - Hoy: `versionCode = 1` / `versionName = "0.1"` hasta el primer release.
-

7. Seguridad

Aspecto	Implementación
Contraseñas	bcrypt cost 12 en ambos lados. El hash se calcula siempre en el dispositivo Android que recibe el texto plano; el endpoint de usuarios recibe <code>password_hash</code> ya calculado y nunca devuelve el hash.
Tokens	JWT <code>HS256</code> , <code>sub</code> = <code>usuario.id</code> , expiración 24 h, firmado con <code>JWT_SECRET_KEY</code> . Sin <code>refresh_token</code> (diferido, ratificado en la Parte 32: se mantiene el JWT de 24 h + re-login al recibir 401, aunque <code>SessionStore</code> persista la sesión para el worker de sync).
Enforcement backend	<code>Depends(usuario_actual)</code> en el router <code>protected</code> : valida firma/expiración y que el <code>sub</code> corresponda a un usuario existente, activo y no borrado. <code>401 {"detail": "token invalido o expirado"}</code> genérico. <code>health</code> y <code>auth</code> exentos.
Autorización por módulo	<code>verificar_modulo</code> gatea solo escrituras (<code>POST</code> / <code>PATCH</code> / <code>DELETE</code>) según <code>(metodo, ruta)</code> -> módulo requerido (mapa en <code>app/permissions.py</code>). Los <code>GET</code> solo exigen token válido. <code>403 {"detail": "el rol no tiene permiso para el modulo: X"}</code> .
Reacción del cliente a 401	<code>AuthInterceptor</code> cierra la sesión y vuelve a <code>LoginScreen</code> solo si la request llevaba token (<code>token != null</code>) y no era <code>/auth/login</code> . Un 401 a una request sin token (esperado en LOCAL / LOCAL_CON_SINCRONIZACION contra un backend con enforcement) lo maneja el repositorio que llamó.
No enumeración de usuarios	<code>/auth/login</code> devuelve el mismo 401 para usuario inexistente, inactivo, sin hash o contraseña incorrecta.
Token de IA	Cifrado AES-256 con clave respaldada por Android Keystore; solo ciphertext + IV en DataStore. El token viaja únicamente en el header del request al proveedor, nunca se loguea.
Autorización de acciones de IA	El módulo a autorizar se deriva del <code>tipo</code> de acción (mapa fijo), nunca del campo <code>modulo</code> de la salida del LLM. Confirmación explícita del usuario para las 4 acciones de escritura.
CORS	<code>CORSMiddleware</code> con <code>allow_origins=["*"]</code> (no hay cliente web; revisar el día que exista uno). Los <code>500</code> del handler global no llevan headers CORS (posición de <code>ServerErrorMiddleware</code> en el stack de Starlette).
Contenedor	Corre como usuario no privilegiado <code>app</code> (<code>adduser --system</code> + <code>USER app</code> en el <code>Dockerfile</code>).
Logging de auditoría	<code>AppLogger</code> (dispositivo): categoría <code>AUTH</code> para logins y rechazos de permiso de la IA; <code>SYNC_CONFLICT</code> para conflictos. Archivos <code>.txt</code> locales, nunca viajan por API. Backend: logging operativo a stdout/stderr, nivel <code>ERROR</code> en el handler global.

Pendientes de seguridad para producción (Partes 36 y 39): APK firmado (hoy `assembleRelease` produce un APK sin firmar), `JWT_SECRET_KEY` fuerte en el entorno desplegado, backend por HTTPS, cleartext

deshabilitado en release, R8/ofuscación (B-7, diferido). La Parte 36 (`docs/PLAN.md`), hallazgos de `docs/review_code.md`) cubre además el endurecimiento previo al release: cambio forzado de la contraseña `admin/admin123` de arranque, arranque del backend sin `JWT_SECRET_KEY` por defecto, límite de intentos de login, `/docs` deshabilitado en producción y `allowBackup` /respaldo de claves de Keystore. Todo pendiente, nada implementado.

8. Referencia de API

Base URL dev: `http://localhost:8000/api/v1`. Detalle completo (bodies, códigos de error, ejemplos): `docs/api-contract.md`.

Método	Ruta	Módulo (escritura)	Notas
GET	<code>/health</code>	— (exenta)	Liveness.
POST	<code>/auth/login</code>	— (exenta)	Devuelve JWT + usuario.
GET / POST	<code>/sucursales</code>	<code>configuracion</code> (POST)	Paginado.
GET / POST	<code>/sync-conflicts</code>	— (infraestructura)	POST idempotente por <code>id</code> del dispositivo.
POST	<code>/ventas</code>	<code>venta</code>	Idempotente por <code>local_id</code> . Decrementa inventario + movimiento.
POST	<code>/entradas</code>	<code>entrada</code>	Idempotente por <code>local_id</code> . Alta de artículo + inventario + movimiento en una llamada atómica.
GET	<code>/inventario</code>	—	<code>sucursal_id</code> requerido; <code>q</code> filtra nombre/SKU/código; <code>updated_since</code> para el pull del sync diferido (Parte 32).
PATCH	<code>/inventario/{articulo_id}</code>	<code>inventario</code>	<code>cantidad</code> = nuevo valor; el server aplica el delta con signo.
GET	<code>/inventario/ubicaciones</code>	—	Valores usados.
GET	<code>/articulos/categorias</code> , <code>/articulos/unidades-medida</code>	—	Valores usados (sin tabla de catálogo).
POST / GET	<code>/cortes-caja</code>	<code>caja</code> (POST)	Idempotente por <code>local_id</code> . Filtros <code>desde</code> / <code>hasta</code> / <code>updated_since</code> (este último para el pull del sync diferido).
GET	<code>/cortes-caja/totales</code>	—	Agrega ventas/retiros del periodo.

Método	Ruta	Módulo (escritura)	Notas
POST / GET	/retiros-efectivo	caja (POST)	Idempotente por local_id. Filtros desde / hasta / updated_since (pull del sync diferido).
POST	/devoluciones	devoluciones	Con líneas. No toca inventario. Idempotente por local_id (Parte 32).
GET / POST / PATCH / DELETE	/roles	usuarios (escrituras)	Roles de sistema: 400 a PATCH/DELETE. DELETE = soft-delete.
GET / POST / PATCH / DELETE	/usuarios	usuarios (escrituras)	Nunca devuelve password_hash. DELETE = soft-delete.

Convenciones: fechas ISO-8601 UTC; IDs UUID v4 (string); paginación page (1-indexed) + page_size (default 20, max 100); errores de negocio { "detail": "... " }; errores de validación con el shape estándar de FastAPI/Pydantic (422).

Al cambiar un endpoint: actualizar primero docs/api-contract.md , luego el backend (schema + router + modelo + migración si aplica), luego el cliente Kotlin (DTO + ApiService + Remote*Repository). Existe una skill sync-api-contract para guiar el proceso.

9. Operación y diagnóstico

9.1 Health y estado

Qué	Cómo
Backend vivo	<code>curl http://<host>:8000/api/v1/health</code> -> <code>{"status":"ok","version":"0.1.0"}</code> .
Contenedores	<code>docker compose ps</code> (backend <code>healthy</code> , <code>migrate</code> <code>exited 0</code>).
Migraciones aplicadas	<code>docker compose exec backend alembic current</code> (debe coincidir con el head del repo, hoy <code>0013</code>).
Prueba de conexión desde la app	Panel de conexión -> "Probar conexión" (<code>BackendHealthChecker</code> , OkHttp propio sin interceptores, contra la URL tecleada).

9.2 Logs del backend

- Módulo `logging` estándar, **nunca** `print()`. Salida a stdout/stderr; el contenedor la captura (`docker compose logs -f backend`). Sin archivos ni rotación propios.
- Nivel `INFO` por defecto; `ERROR` para excepciones no capturadas por el handler global (`app/main.py`), con `exc_info` (traceback completo del lado servidor, nunca al cliente).

9.3 Logs / auditoría del dispositivo

- `AppLogger` escribe en `filesDir` archivos `app-log-<yyyyMMdd-HH:mm:ss>.txt`, una línea por evento: `[TIPO][yyyy-MM-dd HH:mm:ss][sucursalId][usuario] mensaje`.
- Rotación al superar 5 MB (archivo nuevo); purga de archivos con más de 30 días.
- Categorías: `ERROR`, `WARN`, `INFO`, `DB_WRITE`, `SYNC_CONFLICT`, `AUTH`.
- Extracción:

```
adb shell run-as com.pdv.pos ls files/  
adb exec-out run-as com.pdv.pos cat files/app-log-XXXXXXXX-XXXXXX.txt > log.txt
```

- Logcat de red (solo debug): `HttpLoggingInterceptor` nivel `BASIC` (método/URL/estado, sin cuerpos ni headers), gateado por `BuildConfig.DEBUG` (M-8). El cliente de IA nunca tiene interceptor de logging.

9.4 Inspeccionar Room en el dispositivo

Técnica validada (memoria de la Parte 7): copiar `pdv.db` con `run-as` y abrirla con el `sqlite3` de la stdlib de Python (evita depender del binario `sqlite3` en el device):

```
adb exec-out run-as com.pdv.pos cat databases/pdv.db > pdv.db
python -c "import sqlite3;c=sqlite3.connect('pdv.db');print(c.execute('select name from
sqlite_master where type=\"table\"').fetchall())"
```

9.5 Problemas frecuentes y causa raíz

Síntoma	Causa raíz	Acción
El contenedor no refleja un cambio de <code>backend/app/</code>	El <code>Dockerfile</code> hornea el fuente; no hay volumen ni <code>--reload</code> . <code>uv run pytest</code> corre contra el host.	<code>scripts/start-*</code> (hace <code>up -d --build</code>). Verificar con <code>docker compose exec backend</code> .
<code>migrate</code> en <code>restart-loop</code> / esquema inconsistente	Esquema creado fuera de Alembic (<code>create_all</code> sin fila en <code>alembic_version</code>).	Bajar el stack, <code>docker volume rm <proj>_pdv_pg_data</code> , re-levantar para que <code>migrate</code> cree el esquema desde cero; nunca crear tablas fuera de Alembic.
App en modo REMOTO no alcanza el backend por Wi-Fi	Firewall de Windows bloquea inbound TCP 8000 / IP LAN cambió por DHCP / router con client isolation.	Regla <code>New-NetFirewallRule</code> (§5.4); reservar IP o fijarla estática; desactivar AP isolation.
Modo REMOTO funcionó y dejó de funcionar tras reconectar el USB	<code>adb reverse</code> no persiste entre reconexiones/reinicios/ <code>adb kill-server</code> .	Rehacer <code>adb reverse tcp:8000 tcp:8000</code> o pasar a Wi-Fi.
En <code>LOCAL_CON_SINCRONIZACION</code> : crash o logout al vender / entrar a Configuración	Latente desde la Parte 21: <code>GET /sucursales</code> responde 401 sin JWT (login local no tiene token); la <code>HttpException</code> subía sin manejar y <code>AuthInterceptor</code> cerraba sesión ante cualquier 401.	Corregido en la Parte 31: <code>ModeAwareSucursalRepository</code> degrada al catálogo local (<code>.catch</code>); <code>AuthInterceptor</code> solo cierra sesión si la request llevaba token. Si reaparece, revisar que esas dos guardas sigan en su lugar.
Asistente de IA: "El proveedor no devolvió ninguna respuesta" intermitente	DeepSeek <code>deepseek-chat</code> con <code>response_format=json_object</code> devuelve <code>content</code> vacío de forma intermitente (<code>finish_reason</code> / <code>usage</code> en el log de <code>LlmClient</code> lo distinguen).	Conocido (Backlog). Reintentar el mensaje; evaluar <code>max_tokens</code> explícito / reintentado único.

Síntoma	Causa raíz	Acción
Asistente de IA: "La IA devolvió una respuesta con formato inválido"	El proveedor no respetó el JSON de <code>RespuestaIaDto</code> . <code>FORMATO_SALIDA_ACCIONES</code> + <code>instruccionFaq</code> se concatenan al prompt para mitigarlo.	Revisar el log <code>LlmClient</code> (contenido truncado + <code>finish_reason</code>). Verificar el prompt de sistema editable.
Test de backend verde pero el device falla contra el backend dockerizado	pytest corre contra el host; la imagen puede estar desactualizada.	Rebuild de la imagen antes de verificar en device.
Carrera KSP debug/release al compilar Android	Hallazgo transitorio de la Parte 24 (a vigilar en la Parte 26 / CI).	<code>./gradlew clean</code> y recompilar; reportar si es reproducible.
OpenRouter nunca probado en vivo	No hay token de OpenRouter en el entorno.	Cargar un token real para validar antes de recomendar ese proveedor a un comercio.

10. Mantenimiento

10.1 Estructura del repositorio

```
PDV/
├─ android/      App Android nativa (Kotlin)          -> build system Gradle
├─ backend/      API FastAPI (Python)                 -> build system uv + Docker
├─ scripts/      Arranque/detención del backend, tuneling APK;
│               scripts/docs/ (build_manual.py + manual-template.html)
├─ docs/         PLAN.md, PLAN-historico.md, api-contract.md,
│               schema-pos.json, review_code.md (+ la revisión anterior
│               review_code_2026-09-28_1816.md), inventario-inicial.csv,
│               ALCANCE-DOCS.md (alcance del manual), manual-tecnico.md
│               (este archivo) + derivados .html/.pdf
├─ .claude/commands/ Comandos slash del proyecto (/parte, /docs-sync, /jira-sync)
├─ .claude/agents/   Subagentes (doc-maintainer)
├─ .claude/hooks/    docs_guard.py (PreToolUse, registrado en .claude/settings.json)
├─ .github/workflows/ android-ci.yml, backend-ci.yml
├─ docker-compose.yml + docker-compose.prod.yml.example
└─ CLAUDE.md         Contexto de proyecto (raíz del monorepo)
```

Los dos módulos son independientes en build system pero deben mantenerse sincronizados vía `docs/api-contract.md`.

10.2 CI

- `.github/workflows/backend-ci.yml` (trigger en `backend/**`): servicio Postgres 16, `astral-sh/setup-uv@v6` (pin `0.10.12`), `uv lock --check` (falla si `uv.lock` está desactualizado), `uv sync --frozen`, `uv run alembic upgrade head`, `uv run alembic check`, `uv run pytest`.
- `.github/workflows/android-ci.yml` (trigger en `android/**`).
- No hay CI/CD de despliegue todavía (pendiente; Parte 26 del historial / Parte 39).

10.3 Pruebas

Lado	Comando	Estado de referencia
Backend	<code>uv run pytest</code>	15 archivos <code>test_*.py</code> más <code>conftest.py</code> , 170 passed (verificado 2026-09-28, <code>docs/review_code.md</code> §5). Cobertura: happy path + ≥1 error por endpoint, más <code>test_schema_parity.py</code> , <code>test_auth_enforcement.py</code> , <code>test_permisos.py</code> .
Android	<code>./gradlew testDebugUnitTest</code>	90 clases de test; última corrida registrada (cierre de la Parte 32, 2026-09-11; reconfirmada como <code>UP-TO-DATE</code> en <code>docs/review_code.md</code> §5, 2026-09-28): 437 / 0 fallos / 0 errores . ViewModels con estados mockeados; repos locales contra Room in-memory; repos remotos contra MockWebServer; orquestador/pushers/pullers de sync con fakes (sin Robolectric, D5 de la Parte 32).

Ningún PR/feature se considera terminado sin tests unitarios en el lado modificado (`CLAUDE.md` §6). "Compilar" no es evidencia suficiente para cerrar un ítem `jvm-tests`; hay que mostrar la salida del comando.

10.4 Migraciones y `schema-parity`

Un cambio de esquema toca **tres artefactos** que deben quedar alineados (compuerta `schema-parity`):

- `docs/schema-pos.json` — la descripción compartida.
- Entidad Room en `android/.../data/local/*Entity.kt` + migración en `Migrations.kt` + registro en `DatabaseModule` + esquema exportado en `android/app/schemas/`.
- Modelo SQLAlchemy en `backend/app/models/*.py` + migración Alembic nueva en `backend/alembic/versions/` (registrar el módulo en `alembic/env.py` si es una tabla nueva).

Verificación: `uv run alembic check` (drift modelo/migración) y `backend/tests/test_schema_parity.py`.

Estado actual: **13 migraciones Alembic** (`0001` .. `0013`, la última agrega `UNIQUE(local_id)` a devoluciones para la idempotencia del push de la Parte 32), Room en **v8** (línea base v7 con `exportSchema`, más `MIGRATION_7_8` para `cantidadNum`; la Parte 32 no agregó columnas nuevas).

10.5 Añadir o cambiar un endpoint

- Actualizar `docs/api-contract.md` (método, ruta, request, response, errores).
- Backend: `app/schemas/` -> `app/routers/` -> `app/models/` -> migración Alembic si toca esquema -> tests.
- Añadirlo al router correcto (`protected` salvo que deba ser abierto) y, si es escritura con permiso, al mapa `_ESCRITURA_MODULO` de `app/permissions.py`.

4. Android: DTO en `data/remote/dto/` -> `*ApiService` -> `Remote*Repository` -> `ModeAware*Repository` -> `di/NetworkModule` (provee el `ApiService`) -> tests.
5. Rebuild de la imagen del backend; verificación en device si el flujo lo requiere (`needs-device`).

10.6 Convenciones (resumen)

- **Idioma:** documentación y commits en español; **código** (nombres, comentarios, docstrings) en inglés.
- **Kotlin:** `camelCase` funciones/vars, `UPPER_SNAKE_CASE` solo `const val`, `PascalCase` tipos, backing property `_uiState`, booleanos `is/has/should`.
- **Python:** `snake_case`, `PascalCase` clases, esquemas Pydantic con sufijo (`*CreateSchema`, `*ResponseSchema`).
- **Ramas / commits** (`/jira-sync`): `feature/POS-XX-descripcion-corta`, commit `POS-XX: mensaje en imperativo`.
- **Nada de emojis** en código, comentarios, commits, logs ni documentación.
- **No programar a la defensiva:** capturar excepciones solo donde hay un fallo recuperable esperado (red, I/O); preferir que un error no anticipado falle visible.
- **Corrutinas:** concurrencia estructurada, nunca `GlobalScope`.

10.7 Comandos slash del proyecto

- `/parte <n>` — carga la Parte `n` de `docs/PLAN.md` y las secciones que referencia; algunas Partes delegan en subagentes del plugin `feature-dev`.
- `/docs-sync <Parte n> | check` — mantiene este manual (`docs/manual-tecnico.md`) alineado con el código, recorriendo siempre el alcance completo de `docs/ALCANCE-DOCS.md` (nunca parcial). Delega la revisión en el subagente `doc-maintainer` (único que lee y edita el Markdown); tras la confirmación explícita del usuario, genera los derivados `docs/manual-tecnico.html` y `.pdf` con `uv run scripts/docs/build_manual.py` y registra el commit/Parte/ fecha en la línea `docs-sync:` de `docs/ALCANCE-DOCS.md`. `--check` (o `build_manual.py --check`) verifica, sin generar nada, que el sha256 del Markdown coincida con el embebido en el HTML y en los metadatos del PDF. Se corre al cerrar una Parte, antes de `/jira-sync`.
- Hook `PreToolUse` (`.claude/hooks/docs_guard.py`, registrado en `.claude/settings.json` con matcher `Bash|Edit|Write|MultiEdit`): (1) bloquea `Edit` / `Write` / `MultiEdit` sobre los derivados `docs/manual-tecnico.html` / `.pdf` — deben regenerarse con `build_manual.py`, nunca editarse a mano; (2) bloquea `git push` hacia `main` / `master` si el manual quedó desactualizado: hay archivos cambiados fuera de `docs/` desde el commit registrado en la línea `docs-sync:` de `docs/ALCANCE-DOCS.md`, o `build_manual.py --check` falla. `exit 2` con el motivo en `stderr`, visible para Claude.

- `/jira-sync <sección>` — sincroniza una sección de `PLAN.md` hacia Jira (proyecto `P0S`) como épica + historias, unidireccional, con confirmación humana. `PLAN.md` es la única fuente de verdad del alcance. Antes de sincronizar, revisa la frescura del manual contra la línea `docs-sync:` de `docs/ALCANCE-DOCS.md` y sugiere correr `/docs-sync` primero si hay deriva (el usuario puede optar por continuar igual).
-

11. Evolución y deuda técnica conocida

11.1 Estado del plan (docs/PLAN.md)

Las Partes 1-33 están implementadas y mergeadas (su detalle se archivó en docs/PLAN-historico.md); docs/PLAN.md conserva solo las Partes pendientes 34-39, que salen de la revisión pre-release del 2026-09-28/29 (docs/review_code.md , hallazgos A-n / M-n / B-n con reproducción). Ninguna tiene todavía claves de Jira. El checklist completo — criterios de cierre, etiquetas de verificación y decisiones abiertas — vive en docs/PLAN.md ; esta sección solo resume el alcance, no lo duplica.

- **Parte 33 — actualización del manual técnico** (cerrada, PR #42, merge 1ccfc5e): corrigió la numeración de Partes, incorporó el motor de sincronización diferida de la Parte 32, agregó esta sección de estado del plan y revisó el resto del manual contra el estado real del repo. Se decidió (2026-09-11) que su mantenimiento fuera puntual, sin agregar un proceso recurrente a CLAUDE.md / /parte ; la infraestructura de /docs-sync (comando + subagente doc-maintainer + scripts/docs/build_manual.py , §10.7) se agregó después, fuera de la numeración de Partes.
- **Parte 34 — convergencia del inventario en el sync** (pendiente; hallazgos A-1, A-2, M-3; delegada a feature-dev): ajuste de stock por delta idempotente por local_id , inventario.updated_at actualizado en cada movimiento, filas de inventario que dejan de quedar "sucias" tras subir una venta/entrada, cursor del pull por (entidad, sucursal) con ventana de solape y reconciliación de dispositivos ya divergentes.
- **Parte 35 — sucursal estable y rechazos visibles del sync** (pendiente; A-8, A-6, M-5): la sucursal seleccionada no cambia sola al pasar a LOCAL_CON_SINCRONIZACION ; las filas rechazadas con 4xx quedan en un estado propio, registradas en sync_conflicts y contadas en Configuración; un 401 del SyncWorker no cierra la sesión interactiva.
- **Parte 36 — endurecimiento de seguridad pre-release** (pendiente; A-3, A-4, A-5, M-4, M-6, M-10): credenciales de arranque, JWT_SECRET_KEY obligatoria, límite de intentos y /docs en producción, borrado de roles, allowBackup y limpieza de claves obsoletas del DataStore.
- **Parte 37 — integridad de datos del backend** (pendiente; M-1, M-2, B-1, B-2): usuario_id derivado del JWT, validación de montos y fechas, primer insert concurrente de inventario sin 500 , devoluciones sin N+1.
- **Parte 38 — calidad y deuda menor pre-release** (pendiente; M-7, B-3 a B-6, B-8, B-11 a B-13): test MigrationTestHelper de MIGRATION_7_8 , AppLogger , advertencias de lint, textos cortados en el Xiaomi y path_separator de Alembic.
- **Parte 39 — primer release productivo** (pendiente; era la Parte 34 y se renumeró el 2026-10-02): firma de release, despliegue del backend productivo, URL base productiva en la app, endurecimiento del build (R8/shrinking), preparación y distribución, y pantalla principal/tema propios. Detalle en §11.2 (abajo).

11.2 Detalle — Parte 39: primer release productivo (pendiente)

Origen: análisis "qué falta para el primer release productivo" del 2026-09-08, ampliado con los hallazgos A-7, M-9, B-9 y B-10 de `docs/review_code.md` (`docs/PLAN.md`). Grupos:

1. **Firma de release:** `signingConfigs.release` leyendo un `keystore.properties` gitignoreado; keystore generado con `keytool` y respaldado fuera del repo; documentar rotación en `CLAUDE.md` §7.
2. **Despliegue del backend productivo:** destino por decidir (Render / Fly.io / VPS / AWS), `docker-compose.prod.yml` real (`JWT_SECRET_KEY` fuerte, Postgres propio, sin publicar `5432`), TLS válido, migraciones aplicadas; proceso de producción (workers, proxy TLS) y respaldo automático de Postgres con restauración probada (M-9).
3. **URL base productiva en la app:** configurable o horneada; login + sync contra `https://<dominio>` sin `adb reverse`.
4. **Endurecimiento del build de release (B-7):** `isMinifyEnabled = true` + `isShrinkResources = true` + reglas keep para Room, Hilt, kotlinx-serialization, Retrofit, ML Kit. Hoy `isMinifyEnabled = false` (`android/app/build.gradle.kts`).
5. **Cleartext HTTP deshabilitado en release;** allowlist explícito solo si se mantienen pruebas locales.
6. **Preparación del release:** subir `versionCode` / `versionName`, `outputFileName` para el variant release, `testReleaseUnitTest` en verde, `assembleRelease` en el CI (B-9) y decidir si los derivados HTML/PDF del manual se versionan (B-10; hoy están versionados desde el commit `4ff12bb`).
7. **Distribución:** canal (Play Console AAB vs. sideload de APK firmado), flujo de permiso `CAMERA` en runtime con el build de release, y política de privacidad / Data Safety / consentimiento al activar la IA (A-7).
8. **Pantalla principal y tema (A-7):** reemplazar el texto de prueba de la Parte 1, tema propio de la Activity y `core-splashscreen`.

Decisiones abiertas: canal de distribución, destino de despliegue del backend, Play App Signing, política de URL base, expiración de la sesión en modo LOCAL (M-5) y venta por fracción/peso e IVA en el ticket (M-8).

11.3 Seguimiento del motor de sync (Parte 32)

El hueco funcional que describía esta sección — `LOCAL_CON_SINCRONIZACION` sin push ni pull — **se cerró en la Parte 32** (push por entidad vía WorkManager, pull con `updated_since`, `SessionStore` cifrado; ver §2.4). La revisión pre-release (`docs/review_code.md`) encontró, sin embargo, que el inventario no converge entre terminales de la misma sucursal y que el sync tiene fallos silenciosos; esos follow-ups son ahora las **Partes 34 y 35** (§11.1), todas pendientes:

- **Inventario que no converge** (Parte 34, A-1/A-2/M-3): el backend no actualiza `inventario.updated_at` en ventas/entradas, así que el pull no ve esos cambios; en Android la fila de inventario queda con `isSynced = false`; y `InventarioAjustePusher` manda la cantidad absoluta (el `PATCH /inventario` la impone,

pisando ventas de otras terminales). Reproducido en el Xiaomi (12 vs 20, 42 vs 48). El cursor del pull se comparte entre sucursales.

- **Sucursal que cambia sola** (Parte 35, A-8) y **filas rechazadas por 4xx que se descartan en silencio** (Parte 35, A-6, ver política de errores en §2.4). Un 401 del `SyncWorker` hoy cierra la sesión interactiva (M-5).
- **Pull limitado a** `inventario` / `cortes-caja` / `retiros-efectivo` (D3.3): `GET /ventas` y `GET /entradas` no tienen filtro `updated_since` todavía; el catálogo consolidado de `GET /inventario` cubre el caso de uso principal.
- **Filas históricas con** `sucursal_id` **inexistente en el backend actual** quedan `saltadas` permanentemente (dato de antes de que el backend actual existiera; no volverán a intentarse ni bloquean el resto de la cola).
- **MigrationPlanner** / **PlanMigracion** (los 4 casos de migración al cambiar de modo) sigue sin invocarse fuera del catálogo de sucursales (D3.2): la idempotencia por `local_id` de la Parte 23 quitó el riesgo que ese diseño cubría para las entidades transaccionales.

11.4 Backlog (`docs/PLAN.md`)

- **Robustez ante** `content` **vacío del proveedor de IA** (hallazgo Parte 20): reintento único ante `content` vacío y/o `max_tokens` explícito en `ChatCompletionRequestDto`. Afecta el camino compartido de IA, no solo el FAQ.
- **Multi-tenancy compartido** (alternativa evaluada y descartada 2026-08-25): un solo backend/Postgres con `tenant_id` en las tablas en vez de un deployment Docker por franquicia. Reconsiderar solo si administrar N deployments se vuelve caro operativamente.

11.5 Diferidos explícitos

- **refresh_token**: se mantiene el JWT de 24 h + re-login al recibir 401, ratificado en la Parte 32 aunque la sesión ya se persista (`docs/api-contract.md` §13); un dispositivo offline más de 24 h no sincroniza hasta el próximo login interactivo.
- **Política CORS**: revisar `allow_origins=["*"]` el día que exista un cliente web; los `500` del handler global no llevan headers CORS.
- **Persistencia de sesión en el dispositivo**: hoy `SessionManager` es en memoria; se re-logouta en cada arranque. (`SessionStore` , de la Parte 32, persiste la sesión para el worker de sync desatendido, pero `SessionManager` en sí sigue sin sobrevivir un reinicio del proceso fuera de ese camino.)

11.6 Fortalezas a preservar (revisión 2026-08-30, ampliada con la Parte 32)

Separación `domain` / `data.local` / `data.remote` consistente; `ModeAware*` uniforme en las 11 entidades; `EjecutorAccionesIa` sin camino de escritura aparte; login sin enumeración de usuarios; bcrypt cost 12 en

ambos lados; token de IA cifrado con Keystore; POST de sync idempotentes; cobertura de pruebas alta; código sin `!!`, sin `GlobalScope`, sin `Thread.sleep`, un único `runBlocking` justificado. De la Parte 32: los cuatro bugs reales encontrados en la verificación e2e (fila envenenada bloqueando el ciclo completo, falta de feedback de progreso, descarte silencioso de filas irrecuperables, e ids locales sin resolver al remoto en venta/devolución) se encontraron con evidencia directa de dispositivo y base de datos, no solo de logs — ver `docs/PLAN.md` Parte 32 para el detalle completo de causa raíz y fix.

12. Glosario

Término	Significado
<code>BackendMode</code>	Modo de operación del dispositivo: <code>LOCAL</code> , <code>REMOTO</code> , <code>LOCAL_CON_SINCRONIZACION</code> .
<code>ModeAwareRepository</code>	Implementación de una interfaz de repositorio de dominio que delega en la variante local o remota según <code>BackendMode</code> .
<code>local_id</code> / <code>remote_id</code>	PK generada en el dispositivo / PK del backend (<code>id</code>). <code>remote_id</code> es nulo hasta el primer push exitoso.
<code>is_synced</code>	<code>false</code> mientras haya cambios locales sin subir.
<code>deleted_at</code>	Soft-delete: fila inactiva sin borrado físico.
Evento aditivo	Política de sync para cantidades: se combinan deltas con signo (<code>base + deltaLocal + deltaRemoto</code>), nunca se sobrescribe el valor.
<code>last_write_wins</code>	Política de sync para entidades sin ambigüedad de cantidad: gana el <code>updated_at</code> más reciente.
<code>sucursal_scoped</code>	Entidad que lleva <code>sucursal_id</code> (transaccional). Opuesto: catálogos globales (<code>articulos</code> , <code>roles</code> , <code>usuarios</code>).
<code>modulos_permitidos</code>	Lista de claves de módulo que un rol puede ver/usar; activa/desactiva pantallas completas, no acciones dentro de un módulo.
<code>schema-parity</code>	Compuerta que exige alinear <code>schema-pos.json</code> + entidad Room + migración Alembic ante cualquier cambio de esquema.
Parte	Unidad de trabajo del plan (<code>docs/PLAN.md</code> / <code>PLAN-historico.md</code>), con checklist propio.
<code>needs-device</code> / <code>jvm-tests</code> / <code>needs-approval</code>	Etiquetas de verificación de los checklists de <code>PLAN.md</code> .

Estado al 2026-10-02. Archivo generado desde `docs/manual-tecnico.md` (sha256 `76583a0818b7`). No editar este HTML ni el PDF a mano: los cambios se hacen en el Markdown y se regeneran con `uv run scripts/docs/build_manual.py`.